

YAML

```
name: manus-api
description: Manage tasks, projects, and other data in Manus via API, or create
```

Manus API Integration Guide

Use this skill when building or troubleshooting integrations that call the Manus API, automate Manus agents, retrieve generated results, manage projects/files/webhooks/connectors/agents/usage, request structured JSON outputs, or publish websites created by Manus tasks.

This file is the **routing and decision guide**. Keep implementation details in the embedded official docs under `docs/`. When a user asks for exact request bodies, response schemas, validation rules, rate limits, or endpoint-specific edge cases, open the relevant `docs/v2/*.mdx` file instead of relying on this overview.

Version policy. Use **API v2** for all new work. Use `docs/v1/` only when the user is maintaining an existing v1 integration, because v1 is deprecated.

Base URL. Send requests to `https://api.manus.ai` with `x-manus-api-key: <key>`. Successful responses use an `ok: true` envelope; failures use `ok: false` with `error.code`, `error.message`, and `request_id`. API keys are managed from the Manus API integration settings.

1. Documentation Routing

Prefer reading the smallest relevant document before answering detailed implementation questions. For endpoint-specific parameters, open `docs/v2/<endpoint>.mdx`; for the full schema, inspect `docs/v2/openapi_v2.json`.

User intent	Read first
Create, continue, stop, delete, or inspect tasks	<code>docs/v2/task-lifecycle.mdx</code> , then the relevant <code>task*.mdx</code> file
Require JSON output that an application can parse	<code>docs/v2/structured-output.mdx</code>
Publish or manage a website built by an agent	<code>docs/v2/website.mdx</code> , then the relevant <code>website*.mdx</code> file

Upload or reference user files	<code>docs/v2/file.upload.mdx</code> , <code>docs/v2/file.detail.mdx</code> , <code>docs/v2/file.delete.mdx</code>
Configure real-time callbacks	<code>docs/v2/webhooks-overview.mdx</code> , <code>docs/v2/webhooks-security.mdx</code>
Add durable instructions/persona	<code>docs/v2/project.create.mdx</code> , <code>docs/v2/project.list.mdx</code>
Use connectors, skills, agents, or browser clients	<code>docs/v2/connectors.mdx</code> , <code>docs/v2/connector.list.mdx</code> , <code>docs/v2/skill.list.mdx</code> , <code>docs/v2/agents-overview.mdx</code> , <code>docs/v2/browser.onlineList.mdx</code>
Inspect usage, quotas, or logs	<code>docs/v2/usage.list.mdx</code> , <code>docs/v2/usage.teamStatistic.mdx</code> , <code>docs/v2/usage.teamLog.mdx</code> , <code>docs/v2/rate-limits.mdx</code>

2. Endpoint Map

Use this map for orientation only. Open the endpoint file for concrete request/response details.

Group	Purpose	Representative endpoints
Tasks	Agent task lifecycle and multi-turn conversation.	<code>task.create</code> , <code>task.sendMessage</code> , <code>task.listMessages</code> , <code>task.confirmAction</code> , <code>task.stop</code>
Projects	Durable shared instructions and organization.	<code>project.create</code> , <code>project.list</code>
Files	Upload, inspect, and delete files used by tasks.	<code>file.upload</code> , <code>file.detail</code> , <code>file.delete</code>
Webhooks	Production-grade task lifecycle notifications.	<code>webhook.create</code> , <code>webhook.list</code> , <code>webhook.delete</code> , <code>webhook.publicKey</code>
Connectors and Skills	Control third-party app access and skill availability.	<code>connector.list</code> , <code>skill.list</code>
Agents	List, inspect, and update custom IM agents.	<code>agent.list</code> , <code>agent.detail</code> , <code>agent.update</code>

Usage	Retrieve usage records and team statistics.	<code>usage.list</code> , <code>usage.teamStatistic</code> , <code>usage.teamLog</code>
Website	Inspect, publish, version, and update websites built by Manus tasks.	<code>website.status</code> , <code>website.listCheckpoints</code> , <code>website.publish</code> , <code>website.update</code>

3. Core Integration Decisions

Choose the right task flow. For a new independent job, call `task.create` . For an ongoing thread or conversation, store the `task_id` and call `task.sendMessage` on later turns. For direct messaging to the default IM agent, `task.sendMessage` supports the shortcut `task_id: "agent-default-main_task"` .

Use projects for durable behavior. If the integration needs a persistent persona, formatting rule, or reusable instruction, create or reuse a Project and pass `project_id` when creating tasks. Do not duplicate large system-style instructions into every user message if a project is the cleaner abstraction.

Use webhooks for production result delivery. Polling `task.listMessages` is acceptable for scripts and prototypes, but production systems should prefer webhooks and verify signatures with the webhook public key. Still implement idempotency and retries because webhook delivery can be repeated by clients or infrastructure.

Handle waiting states correctly. If the agent asks the user a normal question, answer with `task.sendMessage` . If the agent is waiting for an action confirmation such as sending an email or deploying something, use `task.confirmAction` instead. Read `task-lifecycle.mdx` and `task.confirmAction.mdx` before automating confirmations.

Pick agent profile intentionally. `task.create` and `task.sendMessage` support `agent_profile` values such as standard, lite, and max profiles. Use the endpoint docs for the exact enum and default behavior; note that follow-up turns can override the current profile, while omitting the field keeps the current profile.

Attach tools through message fields. Use `message.connectors` for connector IDs, `message.enable_skills` to control available skills, and `message.force_skills` when a skill must be invoked. Use `connector.list` and `skill.list` to discover IDs.

4. Structured Output Guidance

Use **Structured Output** when the integration needs machine-parseable JSON rather than

human-facing prose. Before writing schemas or code, read `docs/v2/structured-output.mdx` .

Remember only the routing-level behavior here: pass `structured_output_schema` to `task.create` or `task.sendMessage` ; extraction runs after the agent finishes; schemas are **arm once, fire once** and can be re-armed on later turns; results appear in `task.listMessages` as `structured_output_result` or in webhooks as `task_stopped.task_detail.structured_output` . Always check `success` before trusting `value` .

5. Minimal Implementation Pattern

A typical closed-loop integration follows this sequence: upload files if needed with `file.upload` ; create or reuse a project for durable instructions; call `task.create` ; deliver results through webhooks or poll `task.listMessages` ; continue with `task.sendMessage` if the conversation needs another turn; and use `task.confirmAction` only for explicit action confirmations.

For structured-output tasks, add a post-task step that reads either the `structured_output_result` event or the webhook `structured_output` field.

References

Open these files for authoritative details:

Topic	File
Full OpenAPI specification	<code>docs/v2/openapi_v2.json</code>
Task lifecycle and polling	<code>docs/v2/task-lifecycle.mdx</code> , <code>docs/v2/task.listMessages.mdx</code>
Task creation and follow-up turns	<code>docs/v2/task.create.mdx</code> , <code>docs/v2/task.sendMessage.mdx</code>
Structured output	<code>docs/v2/structured-output.mdx</code>
Website publishing	<code>docs/v2/website.mdx</code> , <code>docs/v2/website.status.mdx</code> , <code>docs/v2/website.publish.mdx</code> , <code>docs/v2/website.listCheckpoints.mdx</code> , <code>docs/v2/website.update.mdx</code>
Webhooks	<code>docs/v2/webhooks-overview.mdx</code> , <code>docs/v2/webhooks-security.mdx</code>
File upload	<code>docs/v2/file.upload.mdx</code>

Projects, connectors, skills, agents

`docs/v2/project.create.mdx` ,
`docs/v2/connector.list.mdx` ,
`docs/v2/skill.list.mdx` , `docs/v2/agents-`
`overview.mdx`